

Fo λ ang in Depth

Concepts, Constructs,
and Systems



Kameswara Rao Mithipati

Preface

Why This Book Exists

A programming language is more than syntax; it is the lens through which we view a system's architecture. It shapes how we define boundaries, express state, and whether complexity feels manageable or overwhelming.

FoLang was designed with that in mind, and *FoLang in Depth* was written to examine the reasoning behind its major design choices.

The FoLang Language Reference answers the formal question:

What is FoLang?

It defines the syntax, semantics, type rules, source structure, metadata rules, capability boundaries, and object model. It is the normative specification, and where exact behaviour matters, it governs.

This book answers a different question:

Why are FoLang's major constructs shaped the way they are, and when should you reach for each one?

A specification tells you that a construct exists and how it behaves. It does not always tell you what problem the construct was built to solve, what it costs, which initially similar construct applies to the problem, or what happens when you choose wrongly. That is the space this book occupies.

What This Book Covers

This book examines FoLang's major constructs and consequential design choices, especially the places where several constructs initially appear to solve the same problem. It is not a second language specification or an exhaustive catalogue of every syntactic form. The FoLang Language Reference remains the complete normative source.

For each one, the same questions:

- **What problem does it solve?** Constructs exist for reasons. Knowing the reason usually settles when to use it.
- **How does it behave?** The precise semantics, with the reference cited where the exact rule matters.
- **What does it cost?** Every abstraction has a price — in verbosity, in coupling, in what it forbids elsewhere.
- **What initially appears applicable?** Several constructs may look suitable at first. Once the semantic problem is identified, the applicable choices usually become few.
- **When is it the wrong choice?** Often the most useful section.

Where FoLang offers several constructs of similar nature, the book develops the reasoning once and then shows how it transfers. The goal is not to leave you with a larger menu. It is to help you eliminate choices that do not fit and recognise the one or two that do.

FoLang has a broad vocabulary, but a well-understood problem usually has a narrow answer.

What This Book Assumes

You have written programs in at least one other language and are comfortable with types, functions, and objects.

You do not need to have used FoLang. You do not need a particular background language. Where FoLang differs from what you may be used to, the book says so directly — but it explains FoLang on its own terms rather than as a set of translations from somewhere else.

That distinction matters more than it sounds. Similar syntax does not imply similar semantics, and a construct understood as “the FoLang version of X” tends to be used the way X would be used, which is often not the way it works best.

The FoLang Model

Before the constructs, the model they live in.

FoLang starts from a uniform premise:

Everything in FoLang is an object.

Managed objects use reference semantics by default, so assignment copies a reference rather than duplicating the object's contents. `==` compares values deeply, while `sameRef()` answers the separate question of managed-object reference identity. Mutation, immutability, sharing, copy-on-write behaviour, generic abstraction, matching, functions, and user-defined types live within one model rather than several unrelated ones.

Uniformity does not require one physical implementation strategy.

`co.lang.cstruct` is an explicitly value-semantic ABI representation, because native and foreign interfaces require one. The universal none state has deliberately limited identity semantics, while refinement and dependent values exclude that state. FoLang keeps one conceptual model without erasing the boundaries where different semantics are necessary.

What stays constant is the reasoning. Once you know how FoLang thinks about objects, identity, state, and boundaries, its major constructs become predictable rather than memorised.

A Small Example

Consider two employee objects holding the same values:

```
a Employee = Employee{name: "Rao", id: 1};
b Employee = Employee{name: "Rao", id: 1};

a == b;           // true  - equal values
a.sameRef(b);     // false - different managed objects
```

Think of a shelf holding two identical jars of coffee beans. They contain the same beans, weight, and quantity, so they are equal by value. They are still two different jars.

Now introduce another binding:

```
c := a;
```



```
c == a;           // true
c.sameRef(a);     // true  – the copied reference denotes the same object
```

This does not transfer the jar away from `a`. It gives `c` another handle to the same jar. A mutation through either binding is therefore observed through the other unless another object policy, such as `CopyOnWrite`, establishes isolation.

`==` asks whether the values are equal. `sameRef()` asks whether two bindings refer to the same managed object. `:=` introduces an inferred binding, and its initial assignment follows the ordinary reference-copy rule.

The distinction is not a language quirk. It is part of FoLang's uniform object model. Explicitly value-semantic representations such as `co.lang.cstruct` behave differently, and the book explains where that boundary lies.

How the Choices Narrow

Some FoLang constructs look interchangeable when first encountered. Usually they are not. They answer different questions.

A predicate type, refinement type, and dependent type may initially look like three ways to impose a restriction. The useful question is not which syntax you prefer. It is whether you are constraining candidate types, constraining values, or expressing a type that depends on another value. Once that is known, most of the apparent choice disappears.

The same pattern appears elsewhere:

- a class, interface, trait, and mixin serve different construction, contract, and composition purposes;
- `co.const.none` and a variant alternative communicate different kinds of absence; and
- effect propagation, ordinary error results, and execution-model completion operate at different call and execution boundaries.

The chapters do not present these as equal entries on a menu. They begin with the problem, establish the relevant semantic boundary, remove the choices that do not apply, and then compare what remains.

How This Book Is Organised

Part I — The Model establishes what everything else rests on: objects, bindings, references, equality, mutation, and the boundaries between them.

Part II — Structure covers the constructs that shape a program: structs and companion units, classes, units, modules, and the composition forms.

Part III — Types and Abstraction covers types, generics, interfaces, signatures, type classes and instances, refinement types, and pattern matching.

Part IV — Programs and Systems moves from declarations to systems: packages, project layout, components, libraries and exports, metadata, capability boundaries, and execution models.

Part V — Advanced Facilities covers mechanisms for cases the ordinary abstractions do not reach: dependent types, macros, operators, reflection, specialisation, and native interfaces.

Each part builds on the last, but the major-construct chapters are written to be read individually once the model in Part I is familiar.

Reading the Examples

FoLang uses explicit statement termination. A simple statement ends with `;`, while a block or declaration body ends with its closing `}`.

```
name co.lang.string = "Rao";
age  co.lang.int = 30;
```

```
(co.const.true).then({
  co.out.println("ready");
}).default({
  co.out.println("not ready");
});
```

Examples favour complete, readable forms first. Shorter forms appear once their meaning is established.

Where an example shows something the language rejects, it is marked. Those are often more instructive than the working cases, because a rule is easiest to understand at its boundary.

A Note on Stability

FoLang is approaching version 1.0 but has not reached it. During the alpha period, syntax and semantics may be refined, and examples in this book are updated to track the specification.

Where this book and the language reference disagree, the reference is correct. Report the discrepancy — it means one of the two needs fixing.

Before We Begin

The first chapter does not start with a catalogue of keywords.

It starts with the model, because nearly every design decision in FoLang follows from it:

How does FoLang think about a program?

Once that is in place, the constructs stop being a list to memorise and start being a set of tools with obvious applications.